

Name: Ammar Riaz

Registration Number: 2023377

University: GIKI

Department: FCSE/AI

Course: CS-378 Design and Analysis of Algorithm

Assignment: #1

Semester: Spring 2026

Deadline: 25th Feb, 2026

Problem No 1

1. What will be the time complexity of the above given code and justify it.

- **Time Complexity:** $O(n^2)$
- **Justification:** The provided simplified approach code utilizes two nested loops to evaluate combinations of buy and sell dates. The outer loop iterates from $i = 1$ to n . Inside it, the inner loop iterates from $j = i + 1$ to n . For any given value of i , the inner loop executes exactly $n - i$ times.

To find the total number of operations, we sum the iterations:

$$\text{Total Operations} = \sum_{i=1}^{n-1} (n - i)$$

This is an arithmetic progression that evaluates to $n(n - 1) / 2$, which expands to $n^2 / 2 - n / 2$. When determining asymptotic time complexity, we drop constants and lower-order terms. The dominant term is n^2 , making the worst-case time complexity $O(n^2)$.

2. What will be the time complexity of the above given code both using recursive tree method and substitution/iteration method.

The algorithm FIND-MAXIMUM-SUBARRAY divides the array into two halves of size $n/2$ using recursive calls. It then relies on FIND-MAX-CROSSING-SUBARRAY to calculate the crossing sum, which iterates through the elements in linear time, bounded by $\Theta(n)$. This gives us the recurrence relation:

$$T(n) = 2T(n / 2) + \Theta(n)$$

- **Method A: Recursive Tree Method**

- The root of the recursion tree represents the top-level $\Theta(n)$ cost, which we can write as cn for some constant c .
- At the first level down, the problem splits into 2 subproblems, each taking $c(n/2)$ time. The sum for this level is $2 * c(n/2) = cn$.
- At the second level down, there are 4 subproblems, each taking $c(n/4)$ time. The sum is $4 * c(n/4) = cn$.
- This pattern continues. At depth i , there are 2^i nodes, each contributing $c(n/2^i)$, making the sum at every level exactly cn .
- The problem size halves at each step until it reaches the base case of 1 element. The height of this tree is $\log_2(n)$.
- Summing the cost cn across all $\log_2(n) + 1$ levels gives a total cost of $cn \log_2(n) + cn$. Dropping the constants, the time complexity is $O(n \log n)$.

- **Method B: Substitution Method**

- We will guess the solution is $T(n) = O(n \log n)$ and prove that $T(n) \leq cn \log_2(n)$ for an appropriate constant $c > 0$.
- Let's establish our base recurrence as $T(n) = 2T(n/2) + dn$, where dn represents the linear overhead.
- Assume the bound holds for $n/2$, meaning $T(n/2) \leq c(n/2) \log_2(n/2)$.
- Substitute this assumed bound into the recurrence:

$$T(n) \leq 2(c(n/2) \log_2(n/2)) + dn$$

$$T(n) \leq cn(\log_2(n) - \log_2(2)) + dn$$

$$T(n) \leq cn \log_2(n) - cn + dn$$

- For our initial guess to be correct, we require $T(n) \leq cn \log_2(n)$. This requires the residual part, $-cn + dn$, to be less than or equal to 0.
- This holds true as long as $c \geq d$. Since we can easily pick a constant c large enough to satisfy this, our guess is correct. The time complexity is $O(n \log n)$.

Problem No 2

1. Clearly derive the recurrence relation for the worst-case time complexity.

- The fastPower algorithm utilizes a Divide & Conquer approach.
- In the divide step, it recursively calls fastPower(a, n/2), meaning the algorithm cuts the problem size n by half during each recursive step.
- There is exactly one recursive call made per execution.
- After the recursion returns, the algorithm performs a basic check and executes either one or two multiplication operations. These operations take constant time, $\Theta(1)$.
- Therefore, the recurrence relation for the worst-case time complexity is:

$$T(n) = T(n / 2) + \Theta(1)$$

2. Solve the recurrence using the iteration (substitution) method only.

- Let c represent the constant time $\Theta(1)$ operations overhead. The recurrence becomes:

$$T(n) = T(n / 2) + c$$

- We expand $T(n/2)$ by substituting it back into the equation:

$$T(n) = [T(n / 4) + c] + c = T(n / 4) + 2c$$

- Next, we expand $T(n/4)$:

$$T(n) = [T(n / 8) + c] + 2c = T(n / 8) + 3c$$

- A clear pattern emerges. After k iterative substitutions, the generalized equation is:

$$T(n) = T(n / 2^k) + kc$$

- The recursion bottoms out when the base case is reached. To find out how many iterations it takes to reach a subproblem size of 1, we set $n / 2^k = 1$.
- Solving for k gives $n = 2^k$, which means $k = \log_2(n)$.
- We substitute k back into our generalized equation:

$$T(n) = T(1) + c \log_2(n)$$

- Since $T(1)$ is a constant representing the base case operation, and c is also a constant, we can drop them for the asymptotic notation. The final worst-case time complexity is:

$$O(\log n)$$